



Ficha 14

Recursividad

1.] Introducción.

Cuando en la práctica se habla de *recursividad*, se está haciendo referencia a una muy particular forma de expresar la definición de un objeto o un concepto. Esencialmente, una definición se dice *recursiva* si el objeto o concepto que está siendo definido *aparece a su vez en la propia definición*. Consideremos por ejemplo, la siguiente definición:

Una frase es un conjunto de palabras que puede estar vacío, o bien puede contener una palabra seguida a su vez de otra frase.

Estamos frente a una definición recursiva, puesto que el objeto definido (la *frase*) se está usando en la misma definición, al indicar que una frase puede ser *una palabra seguida a su vez de otra frase*.

Al plantear una definición recursiva debe tenerse cuidado con los siguientes elementos [1]:

1. Una definición recursiva correctamente planteada, exige que la definición *agregue conocimiento* respecto del concepto u objeto que se define. No basta con que el objeto definido aparezca en la definición, pues si sólo nos limitamos a esa regla podrían producirse definiciones obviamente verdaderas, pero sin aportar conocimiento alguno. Por ejemplo, si decimos que:

Una frase es una frase.

no cabe duda en cuanto a que esa afirmación es recursiva, pero de ninguna manera estamos definiendo lo que *es* una frase. En todo caso, lo que tenemos es una *identidad*, y no una *definición*. En cambio en la definición original que dimos de la idea de *frase*, encontramos elementos que nos permiten construir paso a paso el concepto de *frase*, a partir de la noción de *frase vacía* y la noción de *palabra*, y esos elementos son los que agregan conocimiento al concepto.

2. Por otra parte, una definición recursiva correctamente planteada debe evitar la *recursión infinita* que se produce cuando en la definición no existen elementos que permitan cerrarla lógicamente. Se cae así en una definición que, aunque agrega conocimiento, no termina nunca de referirse a sí misma. Por ejemplo, si la definición original de *frase* fuera planteada así:

Una frase es un conjunto que consta de una palabra seguida a su vez de una frase.

tenemos que esta nueva definición es recursiva y aparentemente está bien planteada, por cuanto agrega conocimiento al indicar que una frase consta de palabras y frases. Pero al no indicar que una frase puede estar vacía, la noción de frase se torna en un concepto *sin fin*: cada vez que decimos *frase*, pensamos en una palabra, y en otra *frase*, lo cual a su vez lleva a otra palabra y a una nueva *frase*, y así sucesivamente, sin solución de continuidad.

Las definiciones recursivas no son muy comunes en la vida cotidiana porque en principio, siempre existe y siempre resulta más simple pensar y entender una definición directa, sin la



vuelta hacia atrás que supone la recursividad. La misma noción de frase, puede definirse sin recursividad de la forma siguiente:

Una frase es una sucesión finita de palabras, que puede estar vacía.

y esta definición resulta más natural y obvia (incluso podría no indicarse que la frase puede estar vacía, y la definición seguiría teniendo sentido).

Sin embargo, en ciertas disciplinas como la Matemática, la recursividad se usa con mucha frecuencia debido a que existen problemas cuya descripción simbólica es más compacta y consistente con recursividad que sin ella. Consideremos el típico y ya conocido caso del *factorial* de un número n . Como sabemos, si n es un entero positivo o cero, entonces el *factorial* de n (denotado como $n!$), se puede definir sin recursividad, como sigue:

$$n! = \begin{cases} \text{si } n = 0 \Rightarrow & 0! = 1 \\ \text{si } n > 0 \Rightarrow & n! = n * (n-1) * (n-2) * \dots * 3 * 2 * 1 \end{cases}$$

Es decir: si n es cero, su factorial vale uno. Pero si n es mayor a cero, su factorial es el producto de n por todos los enteros positivos anteriores a n , hasta el uno. De esta forma, el factorial de 4 sería:

$$\begin{aligned} 4! &= 4 * 3 * 2 * 1 \\ 4! &= 24 \end{aligned}$$

La fórmula dada para el cálculo del factorial es sencilla, pero incluye una serie de puntos suspensivos que no siempre son bien recibidos, pues se presupone que quien lee la fórmula será capaz de deducir por sí mismo la forma de llenar el espacio de los puntos suspensivos. Eso podría no ser tan simple si la fórmula fuese más compleja.

La recursividad ofrece una alternativa de notación más compacta, evitando los puntos suspensivos. Por ejemplo, si se observa la definición para $n > 0$, tenemos:

$$n! = n * (n - 1) * (n - 2) * \dots * 3 * 2 * 1$$

En esta fórmula es claramente visible que la expresión $(n - 1) * (n - 2) * \dots * 3 * 2 * 1$ es equivalente al factorial del número $(n - 1)$: se está multiplicando a $(n - 1)$ por *todos los números enteros anteriores a él, hasta llegar al uno*. Por lo tanto, la definición original podría escribirse así:

$$n! = n * (n - 1)!$$

En otras palabras: si $n > 0$, entonces el *factorial* de n es igual a n multiplicado por el *factorial* de $(n - 1)$. Si reunimos todo en una definición global, tenemos:

$$n! = \begin{cases} \text{si } n = 0 \Rightarrow & 0! = 1 \\ \text{si } n > 0 \Rightarrow & n! = n * (n-1)! \end{cases}$$

y llegamos así a una *definición recursiva*: para definir el *factorial* de n , en la misma definición se recurre al *factorial* de $(n - 1)$. Observemos que la condición

$$\text{si } n = 0 \text{ entonces } 0! = 1$$



es la que evita la *recursión infinita*, y que la condición

$$\text{si } n > 0 \text{ entonces } n! = n * (n - 1)!$$

provoca el paso recursivo, pero agregando conocimiento al mismo: un *factorial* es en última instancia un *producto* de un número por el *factorial* del *número precedente*.

2.] Programación recursiva.

La noción de definición recursiva vista hasta aquí sirve como punto de partida para plantear *algoritmos* en forma recursiva (de hecho, la definición recursiva del factorial que se estudió, no es más que un *algoritmo recursivo* para calcular ese factorial).

Prácticamente todos los lenguajes de programación modernos soportan la recursividad, a través del planteo de *subrutinas recursivas*. En Python, la idea es que un algoritmo recursivo puede implementarse a través de *funciones de comportamiento recursivo* [2]. En términos muy básicos, una *función recursiva* es una función que incluye en su bloque de acciones *una o más invocaciones a sí misma*.

En otras palabras, una función recursiva es aquella que se invoca a sí misma una o más veces. En esencia, entonces, la siguiente función es recursiva (aunque aclaramos que está *mal planteada*):

```
def procesar():
    procesar()
```

Si la condición para ser recursiva es invocarse a sí misma, entonces la función anterior cumple el requisito: de hecho, *lo único* que hace es invocarse a sí misma. Sin embargo, como ya se dijo, está mal planteada: aún sin saber mucho respecto de cómo trabaja la recursividad en Python, un breve análisis inmediatamente permite deducir que una vez invocada esta función provoca una *cascada* infinita de auto-invocaciones, sin realizar ninguna tarea útil. Como ya veremos, este *proceso recursivo infinito* provocará tarde o temprano una falla de ejecución por falta de memoria, y el programa se interrumpirá.

¿Por qué está mal planteada esta función? En realidad ya conocemos la respuesta: cuando se introdujo a nivel teórico el tema de las definiciones recursivas bien planteadas en la sección anterior, se indicó que estas deberían *agregar conocimiento* respecto del concepto definido, y evitar la *recursión infinita* incluyendo una condición que corte el proceso recursivo. Y bien: la función *procesar()* aquí planteada no incluye ninguno de los dos elementos. por lo que no corresponde a una definición recursiva bien planteada [2].

Para poner un ejemplo conocido, supongamos que se desea plantear una función recursiva para calcular el factorial de un número *n* que entra como parámetro. Si planteáramos una función *factorial(n)* para calcular *recursivamente* el factorial de *n*, pero lo hiciéramos a la manera incorrecta que vimos antes para *procesar()*, quedaría:

```
def factorial(n):
    return factorial(n)
```

La función *factorial* así planteada, correspondería a "definir" el factorial así:

El factorial de n es igual al factorial de n.



Y como se ve, ni la definición ni la función agregan conocimiento al concepto y son por lo tanto, incorrectas. Un segundo intento, sería:

```
def factorial(n):  
    return n * factorial(n - 1)
```

En este caso, la función mostrada correspondería a definir al factorial de la siguiente forma:

$$n! = n * (n - 1)!$$

lo cual agrega conocimiento pero cae en *recursión infinita*, pues la invocación *factorial(n-1)* provoca una nueva llamada, y esta a su vez otra, y así en forma continua, sin definir en qué momento detener el proceso.

La *recursión infinita* se evita considerando que si $n == 0$, entonces el factorial de n es 1. Una simple condición en la función y queda la versión final, ahora correcta:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

3.] Seguimiento de la recursividad.

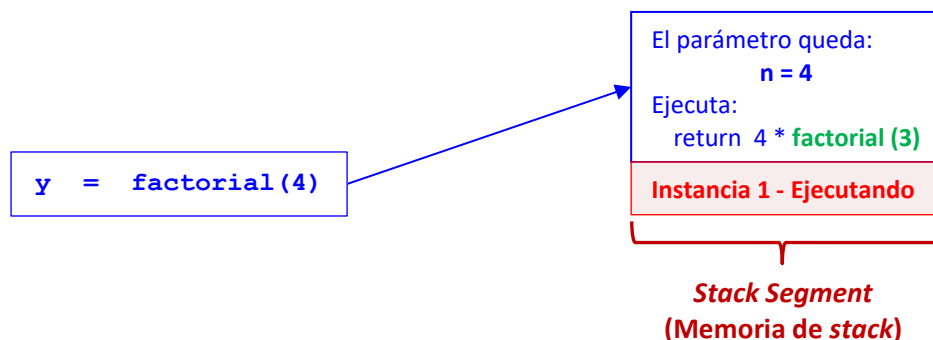
En el punto anterior vimos que una función recursiva bien planteada debe contener una *condición de corte* para evitar la recursión infinita, e incluir una o mas invocaciones a sí misma pero agregando algún tipo de explicitación del proceso. Ahora bien... ¿Cómo funciona todo esto? ¿Cómo hace un lenguaje de programación para ejecutar una función recursiva? Una persona poco entrenada en programación recursiva podría creer que la función *factorial()* que planteamos antes está incompleta (porque no incluye ningún ciclo para calcular el factorial), o asumir que es correcta pero no comprender nada en absoluto respecto de su funcionamiento.

Para entender como trabaja una función recursiva, lo mejor es intentar un seguimiento a partir de un esquema gráfico [1]. Supongamos que se invoca a la función *factorial()* para calcular el factorial del número 4. O sea, supongamos la siguiente invocación:

```
y = factorial(4)
```

Cuando una función es invocada (sea o no recursiva) automáticamente se asigna para ella un bloque de memoria en el segmento de memoria conocido como *Stack Segment* (o *Segmento de Pila*). Dentro de ese bloque, la función crea y aloja sus *parámetros formales y variables locales* y luego comienza a ejecutarse. Se dice que se está ejecutando una *instancia* de la función (y en este caso, es la *primera instancia*).

Recordando que la función *factorial()* toma como parámetro una variable n , en la siguiente figura representamos con un rectángulo al bloque de memoria asignado a la función en esta *primera instancia de ejecución*:

Figura 1: Primera invocación a la función *factorial()*.


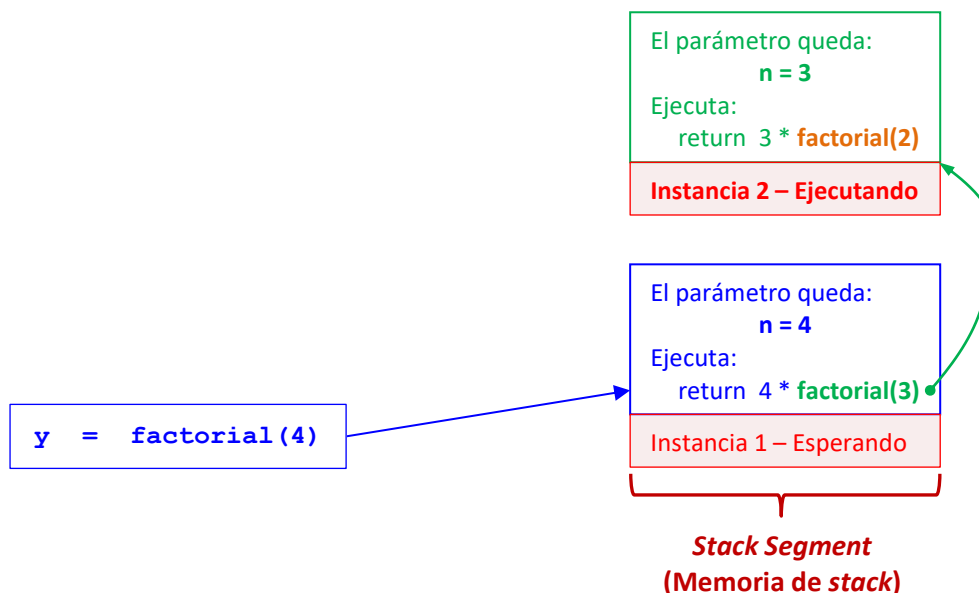
El valor 4 enviado como parámetro actual, es asignado en el parámetro formal *n*. Luego, la función verifica si *n* vale cero. En este caso, la condición sale por falso y ejecuta la rama *else*, que expresa:

```
return n * factorial(n - 1)
```

Como *n* vale 4, la expresión se evalúa como

```
return 4 * factorial(3)
```

Ahora bien: en esta última expresión se está invocando nuevamente a la función *factorial()*, pero ahora para calcular el factorial de 3. *Aquí se produjo una llamada recursiva* y el lenguaje Python actúa frente a ella de manera sencilla: trata a esa invocación recursiva como trataría a cualquier invocación normal de función: simplemente, le asigna a esa *segunda instancia* de ejecución una *nueva área* de memoria de *stack*, para que a su vez esta segunda instancia aloje en ella sus variables locales y parámetros formales. Nuevamente, mostramos un rectángulo para representar a la segunda instancia:

Figura 2: Segunda invocación (recursiva) a la función *factorial()*.


Lo importante aquí es entender que al asignar memoria para la *segunda instancia de ejecución*, la función *vuelve a crear* sus variables locales y parámetros formales, sin interferir con los ya creados para la *primera instancia*. El área de memoria de *stack* asignada a la *primera instancia* sigue ocupada, pero momentáneamente inactiva (la *primera instancia* de ejecución está esperando a que la *segunda* finalice).



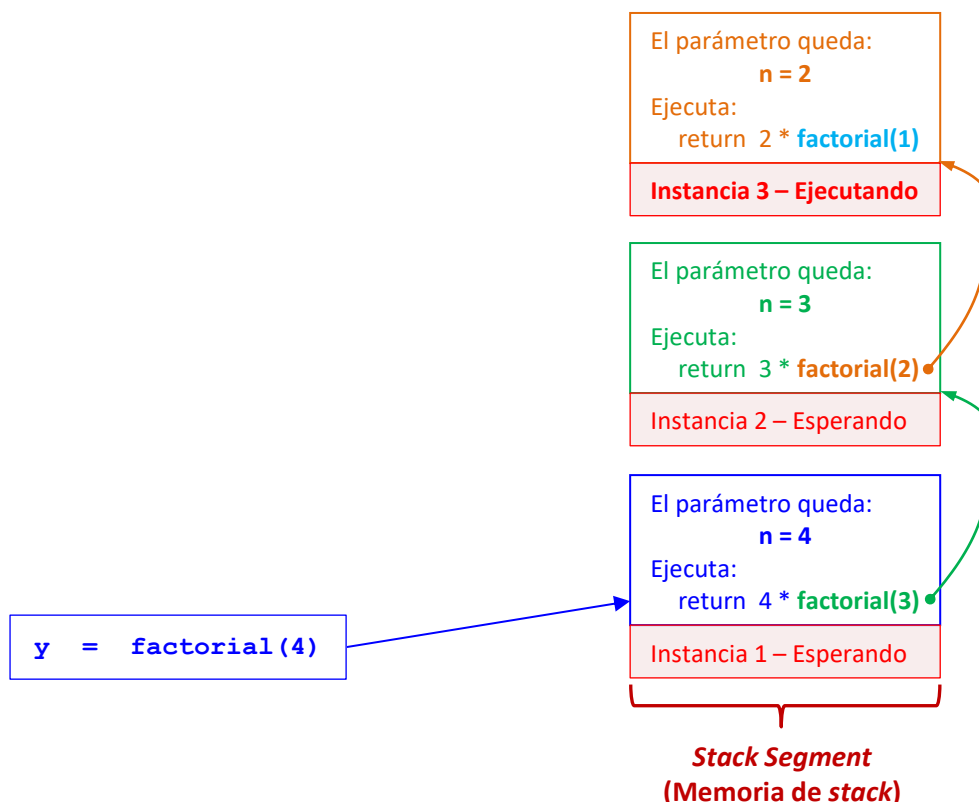
En otras palabras: la *segunda instancia* se comporta (y de hecho, *es*) como una función *diferente* de la primera, y cada una de ellas tiene *sus propias variables*. Aún cuando estas variables tienen los mismos nombres o identificadores en ambas instancias de ejecución, son variables diferentes *porque ocupan lugares distintos de la memoria*. El bloque de memoria de stack de la *primera instancia*, contiene una variable *n* cuyo valor es *4*, y el bloque de memoria de stack de la *segunda instancia* contiene *otra* variable (también llamada *n*), pero con valor igual a *3*. Mientras está activa (o sea, mientras se está ejecutando) la *segunda instancia*, la variable que se usa es la *n* cuyo valor es *3*.

Cuando se ejecuta la *segunda instancia*, se repite el esquema que ocurrió en la *primera*, pero ahora con *n* valiendo 3. La expresión:

```
return 3 * factorial(2)
```

provoca a su vez *otra llamada recursiva*, que es alojada en *otra* área de memoria de memoria de stack. Esta *tercera instancia de ejecución* produce además una nueva creación de variables locales:

Figura 3: Tercera invocación (recursiva) a la función *factorial()*.



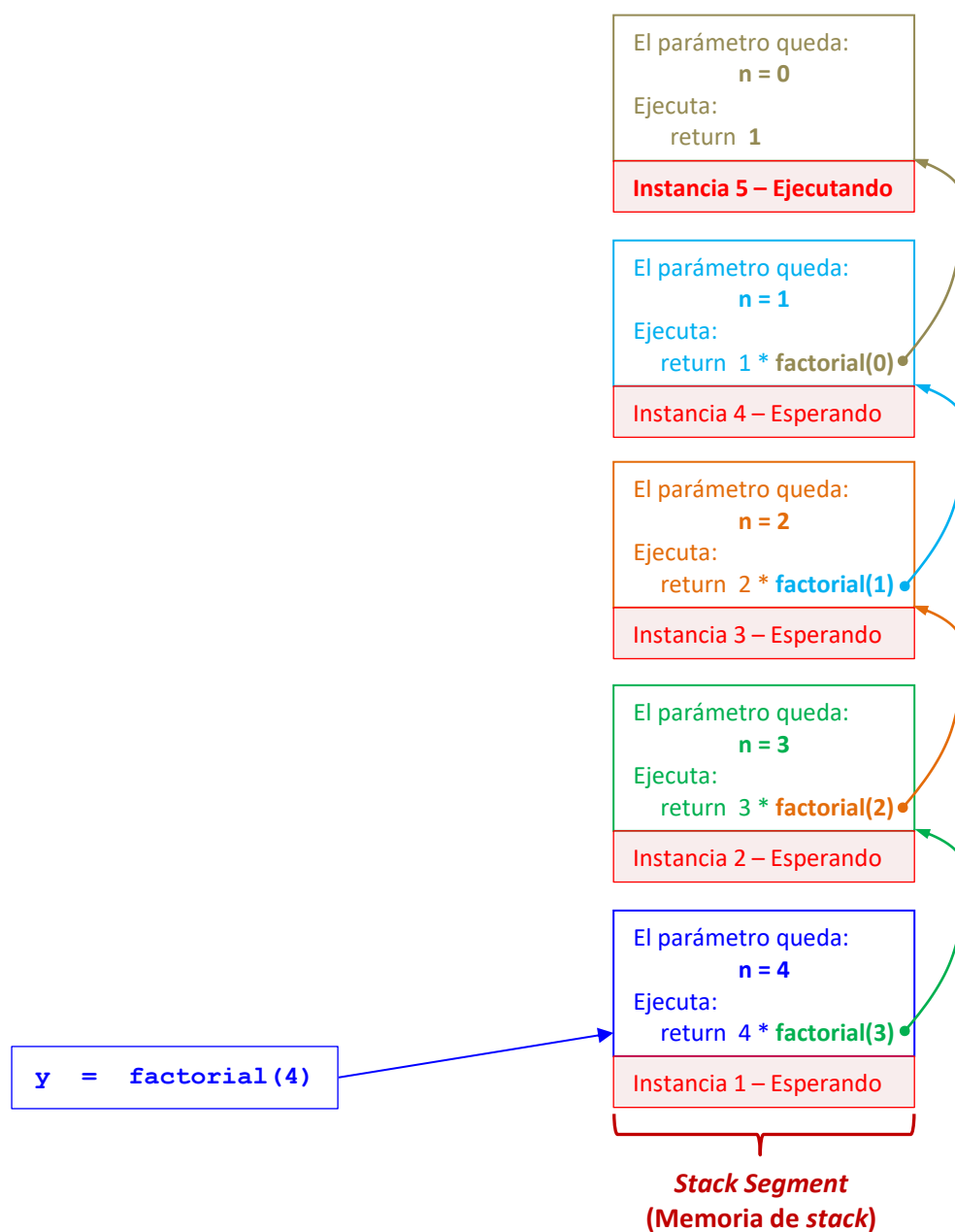
La *tercera instancia* lanza una nueva llamada recursiva y otra asignación de memoria de stack para la *cuarta instancia*, al hacer:

```
return 2 * factorial(1)
```

Y la *cuarta instancia* a su vez lanza una *quinta instancia* haciendo:

```
return 1 * factorial(0)
```

con lo que nuestro esquema de memoria de stack queda así:

Figura 4: Memoria de stack luego de cinco invocaciones recursivas a la función *factorial()*.

Observemos con mucha atención la **quinta instancia de ejecución**: en ella, el valor de *n* es *cero*, y por lo tanto en esta **quinta instancia** la condición:

```
if n == 0:
    return 1
```

es evaluada en *verdadero*, ejecutando entonces la instrucción **return 1**. Entiéndase bien: hasta aquí, había en memoria *cinco instancias* ejecutándose y/o esperando. Si bien se trataba de la misma función invocándose a sí misma varias veces, Python (y todo lenguaje que soporte recursividad) trata a estas instancias como funciones diferentes que piden memoria por separado.



Ahora bien: cuando alguna de las funciones de esa *cascada recursiva* logra finalizar (como pasa con la **quinta instancia** de nuestro último gráfico), comienza a desarrollarse automáticamente un proceso conocido como *proceso de vuelta atrás* o *backtracking*.

La cuestión es simple, aunque un poco extensa de explicar: la función que logra finalizar libera su área de memoria de stack y retorna su valor hacia la instancia de ejecución que originalmente la había llamado. En nuestro caso, la **instancia 5** retorna el valor **1** (que es el valor del *factorial de 0*) hacia la **instancia 4** (que es la que había pedido el *factorial de 0*).

En la **instancia 4** ahora se conoce entonces cuánto vale el *factorial de 0*, por lo cual a su vez puede calcular el valor de $1 * \text{factorial}(0)$ que es igual a **1**. Cuando la **instancia 4** calcula este valor (que es el *factorial de 1*), la **instancia 4** termina de ejecutarse retornando el valor **1** a la **instancia 3** (que fue la que pidió el *factorial de 1*).

En la **instancia 3** se calcula ahora el *factorial de 2*, haciendo $2 * \text{factorial}(1)$ cuyo valor es **2**. Ese resultado se retorna a la **instancia 2**, en donde a su vez se calcula $3 * \text{factorial}(2)$ que vale **6**.

Por fin, el valor **6** es retornado a la **instancia 1**, donde se calcula el valor $4 * \text{factorial}(3)$ cuyo valor es **24** y es justamente el *factorial de 4* que se quería calcular originalmente. Ese resultado es devuelto al *punto original de llamada*, y la función *factorial()* termina (¡por fin!) de ejecutarse. Gráficamente, el *proceso de vuelta atrás* o *backtracking* completo puede verse en la *Figura 5* (página 295).

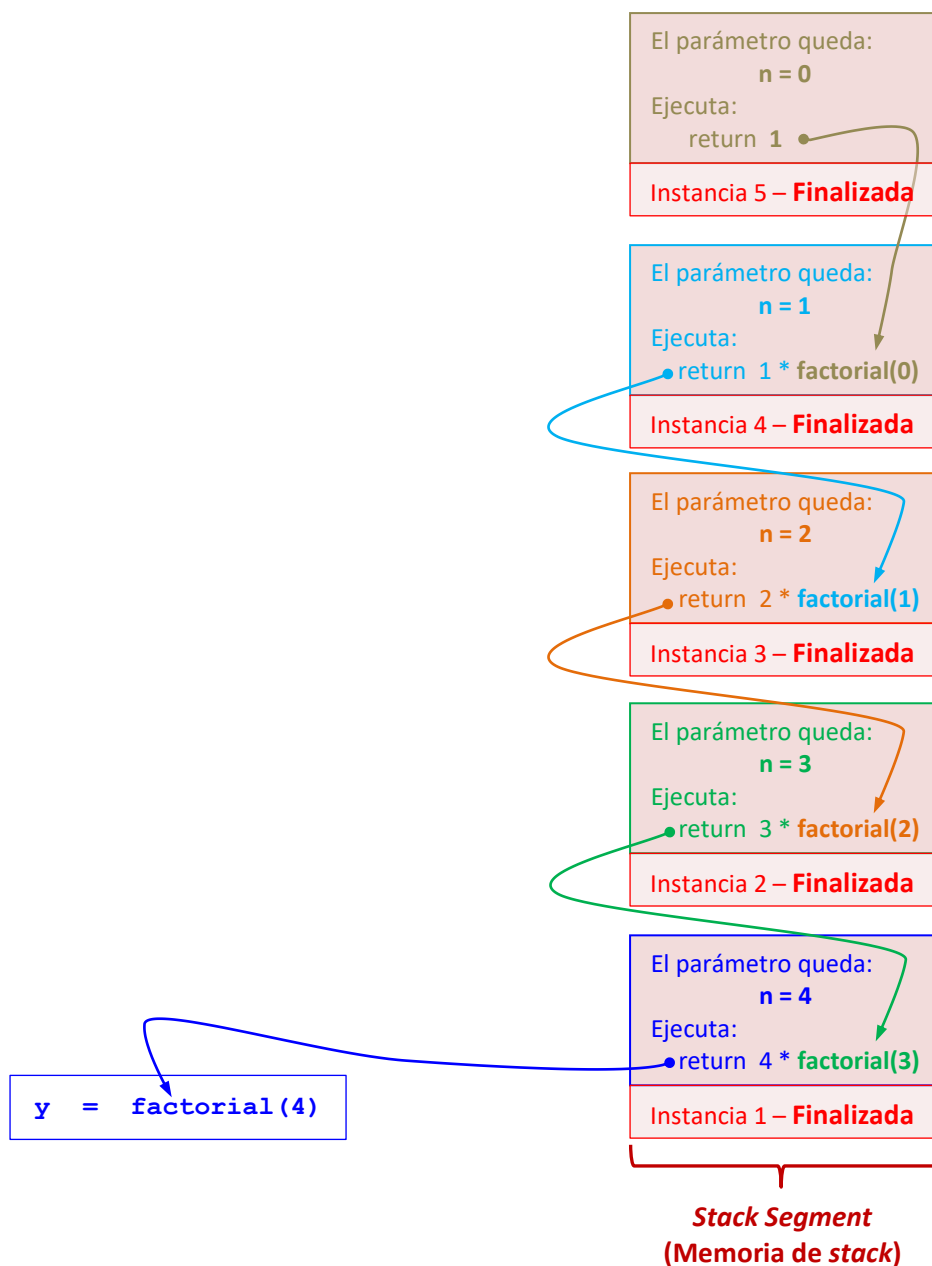
Lo importante de todo esto es que tanto el proceso de asignación de memoria de stack al comenzar el proceso recursivo hacia adelante (o *cascada recursiva*), como el proceso de *vuelta atrás* o *backtracking*, son gestionados en forma automática por el programa. Lo único que debe hacer el programador es *plantear en forma correcta la función*, lo cual como ya vimos, implica incluir una condición de corte y efectuar una o mas llamadas recursivas que de alguna forma vayan reduciendo poco a poco el proceso. En nuestro caso, al llamar a *factorial()* enviando como parámetro el valor $n - 1$, se va reduciendo el valor con el cual se trabaja hasta que en alguna instancia ese valor será cero y comenzará el proceso de *vuelta atrás*.

4.] Aplicación elemental de la recursión.

El proceso recursivo implica una cascada de invocaciones a nuevas instancias de la misma función, y luego otro proceso de regreso o vuelta atrás mediante el cual se van cerrando los cálculos que estuviesen pendientes en instancias anteriores.

El hecho es que no siempre resulta evidente la forma en que ocurre ese proceso de vuelta atrás, sobre todo debido a que el mismo es automático e implícito. Pero si el programador tiene en claro el mecanismo, puede aprovecharlo para lograr resolver problemas que de otro modo parecerían muy complicados. Vemos un ejemplo simple pero muy ilustrativo: Supongamos que se desea implementar una función que simplemente tome como parámetro un número n , y muestre n mensajes en la consola de salida, pero numerados desde 1 hasta n . Supongamos también que se nos pide que la función lo haga en forma recursiva, sin usar un ciclo.

Figura 5: Proceso de *vuelta atrás* completo y finalización de la ejecución.



Un primer intento podría verse como sigue:

```
def mostrar01(n):
    if n > 0:
        print('Mensaje numero', n)
        mostrar01(n-1)
```

Evidentemente, si n es cero o menor que cero, la función no tiene nada que hacer (ya que se están pidiendo "cero mensajes"). Por lo tanto, esa es la *situación trivial* o *base* que define la condición de corte: la función sólo llevará a cabo algún proceso si $n > 0$. Si n es mayor que cero, se muestra el primer mensaje incluyendo en él el valor actual de n , *y luego se activa la recursión*: se vuelve a invocar a la función `mostrar01()` pero ahora pasándole como parámetro el valor $n-1$. Al reducir en 1 el valor del parámetro en cada invocación, se garantiza que en algún momento se alcanzará el valor 0 y la cascada recursiva finalizará.



Todo parecería correcto... pero el estudiante quizás ya notó que hay un problema: así planteada, la función mostrará efectivamente una secuencia de n mensajes, pero numerados en *forma descendente* (de n hacia 1) en lugar de hacerlo en *forma ascendente* (que era lo pedido). Si n es 5, por ejemplo, la salida producida por esta función será:

```
Mensaje numero 5
Mensaje numero 4
Mensaje numero 3
Mensaje numero 2
Mensaje numero 1
```

En efecto: cada instancia de ejecución de la función chequea si n es mayor a 0. Cuando n vale 5 la condición es cierta, y en la rama verdadera la primera instrucción muestra el valor actual de n , **antes** de la invocación recursiva. Por lo tanto, era de esperar que el primer mensaje mostrado se numere como 5 y no como 1. En cada una de las otras instancias de ejecución ocurre lo mismo: primero se muestra el valor del parámetro n , y **luego** de lanza la recursión para los mensajes que quedan.

¿Cómo se podría solucionar el problema y hacer que los mensajes salgan numerados de menor a mayor? Sólo debemos recordar que cada vez que se invoca a la función, se almacena en la memoria de stack el valor del parámetro n . No es obligatorio mostrar el valor de n inmediatamente al entrar en la rama verdadera de la condición: *se puede hacer primero la llamada recursiva*, ir almacenando con eso en el stack la sucesión de valores (sin mostrarlos), y mostrarlos sólo cuando finaliza la cascada recursiva y a medida que se regresa con el proceso de vuelta atrás:

```
def mostrar02(n):
    if n > 0:
        mostrar02(n-1)
        print('Mensaje numero', n)
```

Por curioso que parezca, esta nueva función hace lo pedido y muestra los mensajes numerados en forma ascendente. Lo único que tuvimos que hacer fue *invertir el orden de las instrucciones de la rama verdadera*. Como cada vez que se invoca a la función se almacena en la memoria de stack el valor actual de n , pero estos valores se almacenan de forma que el último en llegar es el que queda disponible en la cima del stack, entonces queda en la cima el valor 1 (puede ver la *Figura 4* para recordar la forma en que se van generando las instancias de ejecución).

Como en nuestra segunda versión las llamadas recursivas se hacen antes que las visualizaciones, el resultado es que **ninguna** visualización se hará hasta que todas las llamadas recursivas queden almacenadas en el stack... y cuando eso ocurra, la primera instrucción `print()` en ejecutarse será la que corresponda a la cima del stack que es la que contiene al valor $n = 1$. El proceso de vuelta atrás hará que se vayan mostrando los valores desde el stack en orden inverso al de su llenado, y eso es lo que hace que los valores se muestren en orden ascendente.

La primera función *mostrar01()* que hemos analizado, tenía una estructura conocida como *procesamiento en pre-orden* o *procesamiento en orden previo*, que se caracteriza por el hecho de que el proceso que debe realizar la función (en este caso, el `print()`), *se hace antes que la invocación recursiva*. Por otra parte, la función *mostrar02()* tiene una estructura que se conoce como *procesamiento en post-orden* o *procesamiento en orden posterior*: el



proceso a realizar (nuevamente la función `print()`) se hace *después de hacer la invocación recursiva*.

Para finalizar, un detalle: el segmento de memoria que hemos llamado *Stack Segment* se designa con ese nombre por una razón de peso: la palabra en inglés *stack* se traduce como *pila* en español y ese nombre se debe a que en ese segmento los valores que se almacenan van *apilándose* uno sobre otro a medida que se van invocando funciones (con recursión o no) (vuelva a ver la *Figura 4* para observar este efecto). Y como ya vimos en una Ficha anterior, esta forma de almacenar datos se conoce como *esquema de Pila* o de *tipo LIFO* por sus iniciales en inglés: *Last In – First Out* (*Primero en Llegar – Último en salir*). El *Stack Segment* es el segmento de soporte para todos los procesos de invocaciones a funciones de cualquier lenguaje de programación. Cada vez que se invoca a una función, se reserva para ella un espacio en la cima del stack y en ese espacio la función almacena la dirección a la que se debe volver cuando finalice su ejecución, y sus variables locales. Y cuando una función finaliza, se libera el espacio de stack que la misma tenía, quedando en la cima la función desde la cual se invocó a la que acaba de terminar. Y es natural que funcione como un *apilamiento* de datos: cuando una función termina, el flujo de ejecución del programa debe regresar al punto desde donde fue invocada la que acaba de terminar, y es justamente esa la que en ese momento estará en la cima. El *Stack Segment* constituye así un mecanismo confiable para que el programa recuerde el camino de regreso que debe seguir a medida que las funciones que se invocaron vayan finalizando.

5.] Consideraciones generales.

Llegado a este punto quizá resulte obvio que la recursividad es un proceso algo complicado de entender para quienes recién se acercan a ella. Sin embargo es también cierto que después de un poco de práctica el proceso se asimila sin inconvenientes, sobre todo si en ese período de práctica el lector se toma el trabajo de hacer un seguimiento gráfico de las funciones recursivas que plantea.

Uno de los puntos que más trabajo parece costar a los recién iniciados es el *planteo de la condición de corte* de la función (es decir, la condición para evitar la *recursión infinita*). La clave de este paso es tener en cuenta que lo que se busca determinar con esa condición es si se ha presentado lo que se conoce como un *caso base* o un *caso trivial* para el problema: un caso en el que la recursión no es necesaria y puede resolverse en forma directa (o incluso no haciendo nada) [1].

La determinación de esa condición de corte puede hacerse sin mayores problemas, simplemente respondiendo a la siguiente pregunta:

¿En qué caso o casos el problema que se quiere plantear se resuelve en forma trivial?

La respuesta a esa pregunta, aplicada al problema particular que se está enfrentando evidencia la condición de corte que se buscaba. En el caso del cálculo del factorial, la pregunta sería: ¿en qué caso el factorial de n se resuelve en forma trivial, sin necesidad de cálculos ni procesos recursivos? Es obvio que ese caso se da cuando n vale cero, pues entonces el factorial vale directamente 1 (uno). Y en nuestra versión recursiva del factorial hemos incluido entonces una condición de corte que comprueba si n es 0.

Por otra parte, debe observarse que la recursividad es una herramienta para usar con cautela [3] [4]: si bien es cierto que una función recursiva es extremadamente compacta en



cuanto a código fuente y permite además que una definición recursiva sea llevada a una función recursiva prácticamente sin cambios (obsérvese la gran similitud que existe entre la definición algebraica recursiva del factorial y el planteo de la función para calcular el factorial), también es cierto que *la recursividad insume más memoria que la que usaría un proceso cíclico común*.

Esto se debe a que cada *instancia recursiva pide memoria de stack* para esa instancia (como se vio en los gráficos anteriores). Si la *cascada recursiva* fuera muy larga (por ejemplo, para calcular el factorial de un número grande), podría llegar a provocarse un *rebalsamiento, desborde o sobreflujo (overflow)* de memoria de stack. Esto significa que alguna instancia de la cascada de invocaciones *podría no tener lugar en el Stack Segment* para ejecutarse, con lo cual se produciría la interrupción o cancelación del programa que invocó a esa función. La memoria de un computador es un recurso de tamaño limitado, y el *Stack Segment* es parte de la memoria.

Por otra parte, la ejecución de una función requiere *cierto tiempo de procesamiento* desde que esta comienza y hasta que finaliza, que debe contemplarse en la estimación del tiempo total de ejecución de un proceso recursivo completo. En el caso del factorial, la versión *iterativa* (no recursiva, basada en ciclos) tendrá un tiempo de ejecución en relación directa con el tiempo que demore el ciclo *for* en finalizar, y este ciclo depende a su vez del valor de n . Por otra parte, la versión *recursiva* hará tantas invocaciones recursivas como sea el valor de n , y el tiempo total dependerá del tiempo que lleve terminar de ejecutar cada una. En este caso, se puede asumir que en ambos escenarios se tendrá un *tiempo de ejecución que depende de n en forma directa*, y por lo tanto serían *igualmente aceptables* la versión recursiva y la versión iterativa.

Notemos además que el código fuente de la versión recursiva es simple, directo y compacto. La *complejidad aparente y la estructura del código fuente* de un programa o función es otro de los elementos que se suelen tener en cuenta para hacer un análisis comparativo entre dos o más soluciones propuestas para un mismo problema. Pero en el caso del factorial, la versión iterativa no es mucho más compleja que la recursiva, aunque es cierto que la recursiva es más directa para comprender.

Si se tienen en cuenta los tres factores generales de análisis comparativo entre algoritmos (*consumo de memoria*, *tiempo de ejecución* y *complejidad de código fuente*) entonces si un programador debe realizar al cálculo del factorial de n , debería usar la *versión iterativa* (y *no la recursiva*). La decisión final de usar la versión iterativa, *se basa en este caso en el uso más eficiente de la memoria por parte de la versión iterativa*. *El tiempo de ejecución final está en el mismo orden de magnitud en ambos casos y la complejidad del código fuente es aceptable también en ambos casos*.

Por estos motivos, en general se suele aconsejar usar la recursividad *sólo cuando sea absolutamente necesario por la naturaleza implícitamente recursiva del problema que se enfrenta*, como es el caso del recorrido de ciertas estructuras de datos como los *árboles* o los *grafos*, o la generación de gráficos de *naturaleza fractal* (figuras que se forman combinando versiones más sencillas de las mismas figuras...) que resultaría muy difícil de programar sin recursión desde el punto de vista de la *complejidad del código fuente*.

Algunos programadores experimentados a veces plantean primero la solución recursiva de un problema (para darse una idea de la forma mínima de resolverlo desde el punto de vista



de la *complejidad del código fuente*), y luego tratan de convertir ese planteo a una solución no recursiva basada en ciclos (aunque esto no siempre es sencillo de hacer...)

6.] Caso de análisis: La Sucesión de Fibonacci.

Para permitir un mayor dominio de las técnicas de programación recursiva, y sus ventajas y desventajas, analizaremos ahora un conocido problema: el cálculo del término n -ésimo de la *Sucesión de Fibonacci*.

Problema 34.) *La Sucesión de Fibonacci es una secuencia de valores naturales en la que cada término es igual a la suma de los dos anteriores, asumiendo que el primer y el segundo término valen 1. En algunas fuentes se parte de suponer que los dos primeros valen 0 y 1 respectivamente, pero eso no cambia el espíritu de la regla ni la explicación que sigue. Formalmente, el cálculo del término n -ésimo $F(n)$ de la sucesión se puede entonces definir así (y note que la definición planteada es directamente recursiva):*

Término n -ésimo de Fibonacci

$$F(n) \quad \left\{ \begin{array}{ll} = 1 & (\text{si } n = 1 \text{ ó } n = 0) \\ = F(n-1) + F(n-2) & (\text{si } n > 1) \end{array} \right.$$

(n entero, $n \geq 0$)

Se pide desarrollar un programa que permita calcular el valor del término n -ésimo de esta sucesión, cargando n por teclado.

Discusión y solución: Si bien el enunciado muestra la definición en forma directamente recursiva, podemos diseñar un par de funciones que calculen el valor del término n -ésimo de Fibonacci: una en forma iterativa y la otra en forma recursiva para luego poder comparar ambas soluciones (vea el proyecto *F[26] Recursión* que acompaña a esta ficha):

```
# versión iterativa
def fibonacci01(n):
    ant2 = ant1 = 1
    for i in range(2, n + 1):
        aux = ant1 + ant2
        ant2 = ant1
        ant1 = aux
    return ant1

# versión recursiva
def fibonacci02(n):
    if n <= 1:
        return 1
    return fibonacci02(n - 1) + fibonacci02(n - 2)
```

La *versión iterativa* inicializa las variables *ant2* y *ant1* con el valor 1, y usa un *for* para recorrer el intervalo $[2, n]$. En cada repetición suma los valores de *ant1* y *ant2* para obtener el siguiente término en forma progresiva, actualizando también los valores de *ant1* y *ant2* para quedarse siempre con los dos últimos calculados. Cuando el *for* finaliza, el último número almacenado en *ant1* es el que corresponde al término n -ésimo pedido.

La *versión recursiva* es la aplicación directa de la fórmula dada en la definición: la condición de corte comprueba si n es menor o igual a 1. En caso afirmativo, se asume que n vale 0 o

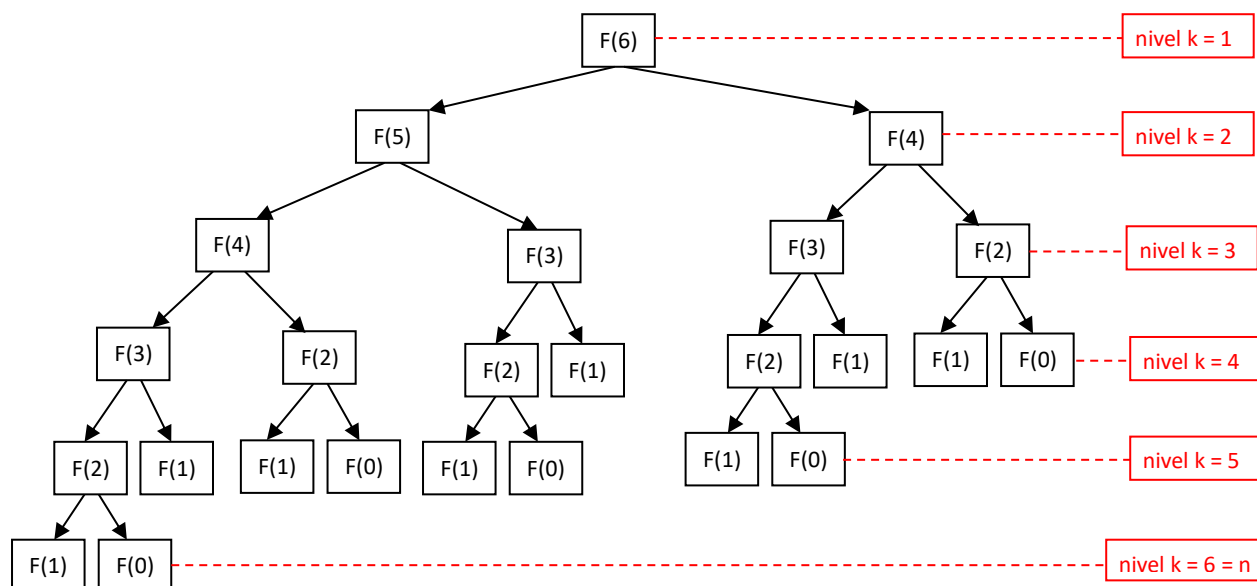
vale 1 y en ese caso, se retorna 1 como se pide en la definición. Si n fuese mayor a 1, se aplica la fórmula en forma recursiva y se retorna la suma de los dos anteriores a n .

Está claro que la *versión iterativa* se ejecuta en un tiempo proporcional a n , en forma similar al cálculo del factorial: a medida que n aumenta, en la misma proporción aumenta el tiempo de ejecución ya que depende de cuántas repeticiones haga el ciclo *for*. Por otra parte, el espacio de memoria empleado por la *versión iterativa* es siempre el mismo: un puñado de variables locales, que no cambia aunque n sea mayor o menor.

Si se comienza un análisis comparativo, está muy claro que la *versión recursiva* es evidentemente más simple de entender y más compacta en cuanto a *complejidad de código fuente* que la *versión iterativa*. Pero el análisis del *tiempo de ejecución* (y en menor medida, del *consumo de memoria*) de la *versión recursiva* no es tan simple debido a la presencia de **dos** invocaciones recursivas.

Para intentar una aproximación intuitiva, podemos mostrar un gráfico de la estructura de la *cascada de llamadas* que implica la *versión recursiva*. Si suponemos que se desea calcular el valor de *fibonacci02(6)* (que abreviaremos como $F(6)$ en el gráfico, para simplificar), se puede ver que se producirá un *árbol de llamadas* similar al siguiente (aunque comprenda, *no todo el árbol estará contenido en la memoria al mismo tiempo*):

Figura 6: Esquema del árbol de llamadas recursivas para $F(6)$.





Pero los problemas verdaderamente comienzan cuando se analiza el *tiempo de ejecución*. En este caso, **sí** importa el *árbol completo*, pues debemos calcular *cuánto tiempo llevará hacer el proceso total*. De nuevo, el hecho técnico de invocar a una función lleva un tiempo constante, pero a ese tiempo debe sumarse el que lleve completar el proceso contenido en ella. En nuestra **versión recursiva**, el proceso consta básicamente de una suma que también ejecuta en tiempo constante... y por lo tanto, *todo queda reducido a saber cuántas veces se invoca la función a lo largo de toda la ejecución*.

Se puede sospechar que será un número elevado, ya que sólo para **$F(6)$** tenemos 25 invocaciones (el número de rectángulos o *nodos* del árbol anterior), pero además podemos ver que muchas de esas invocaciones se hacen para realizar un proceso que ya había sido completado antes: sin contar los cálculos triviales para $F(1)$ y $F(0)$, vemos que dos veces se calcula $F(4)$, tres veces se calcula $F(3)$ y cuatro veces se calcula $F(2)$. El árbol no sólo es denso de por sí, sino que además se pierde tiempo re-calculando valores que ya se tenían...

En definitiva: ¿cuánto tiempo llevará el proceso completo para **$F(n)$** en la **versión recursiva**? Intuitivamente, si se observa el árbol de llamadas recursivas para **$F(6)$** , puede verse con claridad que el número total de invocaciones a medida que se completa cada nivel aumenta en forma más o menos predecible:

Nivel	Invocaciones hasta ese nivel (incluido)
$k = 1$	$1 = 2^1 - 1$
$k = 2$	$3 = 2^2 - 1$
$k = 3$	$7 = 2^3 - 1$
$k = 4$	$15 = 2^4 - 1$
$k = 5$	23
$k = 6$	25

Al menos hasta el nivel $k = 4$, todos los niveles están completos y el número de llamadas acumulado hasta allí varía en *forma exponencial*. Y con ello podemos empezar a suponer que el tiempo de ejecución será algo de la forma b^n para alguna base b constante y mayor a 1.

Puede probarse que este resultado intuitivo es efectivamente correcto, y veremos que si un algoritmo/programa tiene un tiempo de ejecución exponencial (como en este caso) entonces estamos en serios problemas... Si el tiempo de ejecución de un algoritmo cambia en relación exponencial a medida que n crece, entonces ese algoritmo sólo será aplicable cuando n sea realmente muy pequeño.

Si n toma un valor de entre 30 y 35 la cantidad de pasos que el algoritmo supone es tan grande que incluso una computadora moderna muy veloz comenzará a verse en problemas para terminar el proceso y entregar el resultado en un tiempo prudente. Puede probar el programa `test01.py` del proyecto [F14] *Recursión* que acompaña a esta ficha, y cargar los valores 30, 31, 32, 33, 34 y 35 para darse una idea de lo que ocurre con pequeñas variaciones en el valor de n ...

Para valores de n mayores a 35 y hasta 40 el proceso se torna penosamente lento, de forma que ya para $n = 40$ posiblemente tenga que detener el programa en forma manual (y todo mientras la **versión iterativa** finaliza en un instante).

Para $n = 50$ o $n = 60$ el **proceso recursivo es definitivamente inaceptable** en términos prácticos: la computadora podría estar miles o centenares de miles años haciendo el cálculo



y buscando el resultado (que la *versión iterativa* obtiene en unos pocos milisegundos). Y si fuese $n = 100$, entonces será hora de rendirse: el *programa recursivo* ejecutará *durante todo el tiempo de vida que le queda al universo, y no alcanzará a calcular el valor de $F(100)$* . Ejecute el programa *test01.py* ya mencionado, e inténtelo si no lo cree...

Si un problema *solo admite soluciones que se ejecutan en tiempo exponencial*, entonces ese problema se dice **intratable** y constituye todo un desafío para las Ciencias de la Computación. Note que el cálculo del término n -ésimo de Fibonacci **no es un problema intratable**, ya que se conoce al menos un *algoritmo de tiempo de ejecución no exponencial* para resolverlo: nuestra ya conocida *versión iterativa*.

A modo de conclusión:

- La recursividad es una herramienta muy útil para el planteo de algoritmos, pero debe ser usada con cuidado y con conocimiento adecuado en cuanto a la forma de estimar el uso de recursos de tiempo y memoria.
- En general, desde el punto de vista de la *complejidad del código fuente*, la recursión permitirá escribir programas más compactos, más simples de comprender, y más consistentes con la definición formal del problema que en un planteo iterativo. Pero si se toma a la ligera el consumo de tiempo y memoria usada, el programa obtenido podría simplemente ser inaceptable en la práctica.
- Algunos programadores suelen ensayar soluciones recursivas para un problema, dado que en términos de complejidad de código fuente podría resultar más simple; y si luego descubren que esas soluciones son poco eficientes proceden a eliminar la recursión y convertir el programa en un proceso iterativo (aunque como dijimos, no siempre esto es simple de hacer)
- ¿¿¿Obtuvo ya el resultado de $F(100)$???
- A la luz de todo lo dicho, podría alegarse que entonces no es conveniente aplicar recursividad *en ningún caso*. Sin embargo, esta postura pesimista está lejos de ser cierta: si la recursión se aplica en la forma apropiada, puede dar lugar a soluciones muy eficientes (al menos en cuanto a tiempo de ejecución) y sorprendentemente compactas en cuanto a complejidad de código fuente. Ciertas estrategias de resolución de problemas (como la que se conoce como *Divide y Vencerás (DyV)*) se basan en un esquema recursivo aplicado con ciertas restricciones, y de acuerdo a esas restricciones pueden resolverse distintos problemas en forma muy eficiente respecto del tiempo de ejecución.
- La estrategia conocida como *Backtraking* (o *Vuelta Atrás*) es otra estrategia de planteo de algoritmos que se monta sobre un proceso recursivo para ayudar a explorar diversas soluciones a un problema y quedarse con la mejor a medida que las instancias recursivas van finalizando. Muchos de los problemas que se resuelven con *Backtraking*, serían casi imposibles de resolver sin esa técnica.
- Sin embargo, el enfoque pesimista todavía podría alegar que las estrategias *DyV* y *Backtraking* son *técnicas diferentes* y con *entidad propia* para el planteo de algoritmos. Se podría alegar que la *recursividad aplicada en forma directa* y sin las restricciones propias de la estrategia *DyV* o el *Backtraking* no es práctica. Y de nuevo, la respuesta es que eso no es cierto. Como dijimos, muchas aplicaciones propias del recorrido de estructuras de datos no lineales (como *árboles binarios*, *árboles n -arios* y *grafos*) se resuelven en forma natural y eficiente usando *recursividad directa*, con el adicional de que esos procesos resultan muy compactos y simples de comprender en cuanto a código fuente.
- ¿¿¿Obtuvo ya el resultado de $F(100)$???

- Y por supuesto, existen áreas específicas en las ciencias de la computación en las que el empleo de la recursión en forma directa o indirecta (la *recursión indirecta o mutua*, es aquella situación en la que una función $A()$ invoca a otra $B()$, pero luego $B()$ invoca a su vez a $A()$) es prácticamente obligado, pues de otro modo los algoritmos serían absurdamente difíciles de plantear. Una de esas áreas es la de la generación de *figuras fractales*. En casos así, el factor de eficiencia preponderante es evidentemente, la *complejidad del código fuente*.
- Ya puede cancelar el programa del cálculo de $F(100)$. A menos que esté dispuesto a esperar hasta el fin de los tiempos....

7.] Aplicaciones de la recursividad: Introducción a los gráficos fractales.

Hemos indicado que la recursividad debe ser usada con cuidado ya que aplicada con criterio incorrecto puede llevar a situaciones de excesivo (e innecesario) consumo de memoria (como se vio para el caso del factorial), o bien a implementaciones cuyos tiempos de ejecución resultan inaceptables (como vimos para el caso de la sucesión de Fibonacci).

Pero también hemos indicado que ciertos casos generales la recursión es una excelente herramienta para planteo de soluciones a problemas que de otro modo serían extremadamente difíciles desde el punto de vista de la complejidad del código fuente. En este tipo de problemas, el consumo extra de memoria que hace la recursión está justificado y el tiempo de ejecución será aceptable.

Como ya se dijo, una de las áreas en que la recursión es recomendable es el procesamiento de *estructuras de datos no lineales* como los *árboles* o los *grafos*. Y otra de esas áreas es la *generación y tratamiento de figuras y gráficos fractales*, que es el tema sobre el que está centrado el desarrollo de esta sección. [1] Una *figura fractal* es aquella que se compone de versiones más simples y pequeñas de la misma figura original, y curiosamente existen numerosos ejemplos de formaciones fractales en la naturaleza, como se ve en las imágenes de la figura que sigue:

Figura 7: Algunos ejemplos de formaciones *fractales naturales*.¹



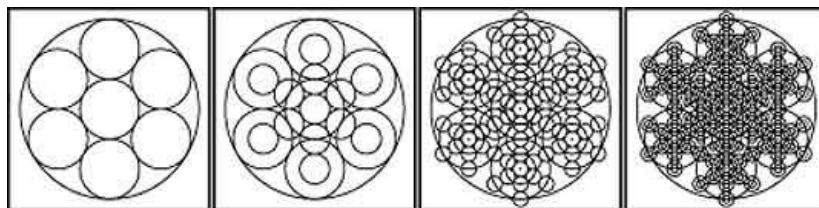
Muchas figuras fractales naturales pueden imitarse y reproducirse artificialmente mediante algoritmos recursivos. Pero también puede pensarse directamente en generar figuras y

¹ Las fuentes de las imágenes que se muestran en la Figura 1 son las siguientes, tomadas de izquierda a derecha:

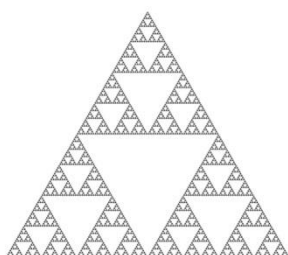
- ✓ <http://ztfnews.files.wordpress.com/2010/10/helechom.gif>
- ✓ http://www.oddee.com/media/imgs/articles/a302_f5.jpg
- ✓ <https://cuquialcocer.files.wordpress.com/2012/04/fractal2.jpg>

curvas basadas en geometría fractal que no estén inspiradas en figuras naturales². La *Figura 8* provee algunos ejemplos relativamente simples de visualizar:

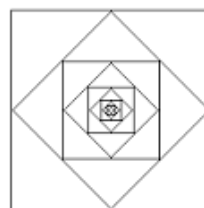
Figura 8: Algunas gráficas *fractales artificiales* simples.³



a.) *Composición fractal con círculos*



c.) *Composición fractal con triángulos*



b.) *Composición fractal con cuadrados*

Todas la gráficas de la *Figura 8* están compuestas por la misma figura geométrica, repetida una y otra vez con diversos tamaños y posiciones siguiendo un patrón. Cada imagen remite de inmediato a una secuencia recursiva gráfica (y cada una puede entenderse como una definición recursiva intuitiva de la forma "un círculo compuesto por círculos" o un "cuadrado compuesto por cuadrados", etc.) Y como veremos, esta clase de figuras son especialmente aptas para dibujarse mediante programas recursivos (las gráficas **b** y **c** de la figura anterior están programadas en el proyecto [F14] *Piramide* que acompaña a esta Ficha [programas [caleidoscopio.py](#) y [triangulo.py](#)])

8.] Generación de gráficos fractales.

La misma idea que se expuso en el apartado anterior vale para la atractiva pirámide coloreada que se muestra en la *Figura 9*. La figura es una composición fractal formada exclusivamente por cuadrados, de manera que se vayan dibujando unos sobre otros para armar los cuatro soportes de una estructura piramidal vista desde arriba: El gran cuadrado

² Las figuras fractales naturales también son comunes en el cuerpo humano y un ejemplo es el iris del ojo en cuanto la distribución del color y las formaciones internas que pueden observarse en él. Al respecto, una extraña película del año 2014, titulada *I Origins* (conocida en español como *Orígenes*), dirigida por *Mike Cahill* y protagonizada por *Michael Pitt*, *Àstrid Bergès-Frisbey* y *Brit Marling* narra la historia de un investigador de la evolución del ojo humano que hace un descubrimiento colateral sorprendente, justamente en relación al iris... Extraña pero digna de verse, prestándole atención a los detalles.

³ Las fuentes de los gráficos que se muestran son las siguientes:

a.) <http://www.thecamino.com.ar/imagf/spiral08.jpg>

b.) http://www.infinitefractal.com/movabletype/blogs/my_blog/cart/Diapositiva3.jpg

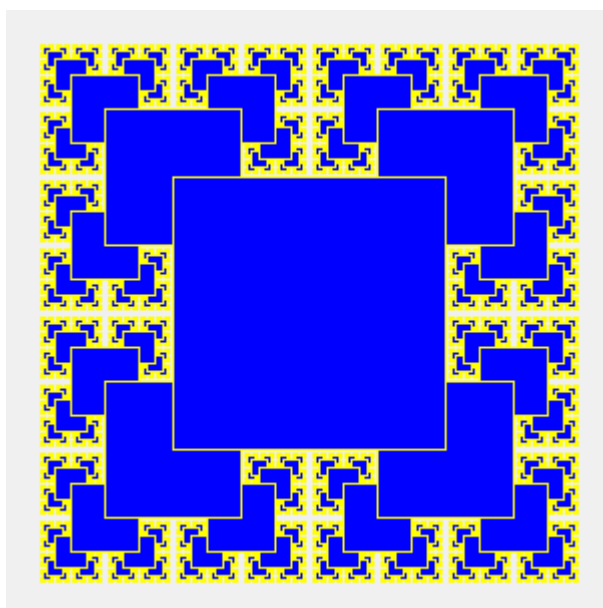
c.) <http://2.bp.blogspot.com/-AD-lxWAAzg/Ughm1btCCPI/AAAAAAAAA40/1NBDGQIGhO0/s1600/fractal+6.png>

azul que se ve en primer plano constituye la cima de la pirámide, y los cuatro soportes se van estirando hacia los cuatro vértices de la gráfica. [1]

La figura no sólo es fractal por estar compuesta por cuadrados que van disminuyendo su tamaño: cada uno de los soportes es a su vez otra pirámide más pequeña, con exactamente la misma forma y proporciones que la pirámide completa.

Esto comienza a sugerir la idea de usar una función recursiva para lograr el dibujo entero. La parte superior de la pirámide muestra un gran cuadrado a modo de tapa, y otros cuatro cuadrados más pequeños (con lados la mitad de largos que el cuadrado mayor) sirviendo como apoyo en cada uno de los vértices. A su vez cada uno de esos cuatro cuadrados tiene otros cuatro más pequeños como soporte, y la idea se repite hasta la base de la pirámide.

Figura 9: *Pirámide fractal* formada por una composición de cuadrados coloreados.⁴



Python provee numerosos y potentes mecanismos para el diseño de ventanas y componentes gráficos. Si bien no forma parte de los objetivos de este curso hacer un estudio detallado respecto de esos mecanismos, el hecho es que resulta relativamente simple desarrollar un programa que genere mínimamente gráficos como el anterior. [2] [3]

Además, también es cierto que un curso de programación introductorio podría verse muy beneficiado si los estudiantes pudieran utilizar recursos visuales de alto impacto en el corto plazo: la experiencia nos ha mostrado que esos recursos incentivan la creatividad y constituyen un fuerte impulso de motivación: al fin y al cabo, resulta obvio que un programa resulta mucho más atractivo de usar (y de desarrollar...) si el mismo cuenta con ventanas y gráficos que hagan más sencillo su funcionamiento y más accesibles sus recursos. [4]

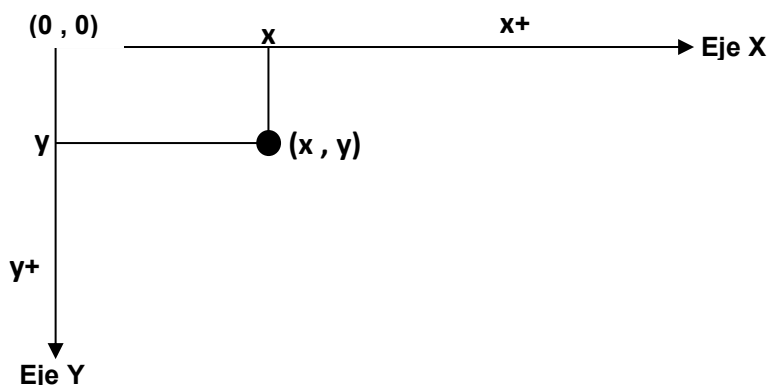
Por lo pronto, el primer elemento que debe quedar claro y dominarse rápidamente es el *esquema de coordenadas de pantalla* que Python (y todo otro lenguaje) supone para la

⁴ La idea de la pirámide fractal expuesta en esta sección está inspirada en la presentación del tema en el libro "Estructuras de Datos en Java" de Mark Allen Weiss (página 179).

gestión de gráficos. El sistema de coordenadas de Python permite localizar a cada punto de luz o *pixel* de la pantalla en forma directa.

Por defecto, el punto superior izquierdo dentro del área de dibujo de una ventana o un lienzo gráfico (conocido como un *canvas*) es el *origen de coordenadas* de ese contenedor gráfico (o sea, el punto de coordenadas $(0, 0)$ del contenedor). Si la distancia de un *pixel* al origen del sistema en el eje horizontal se designa como x , entonces x será la *coordenada de columna* de ese pixel. De forma similar, si la distancia del pixel al origen en el eje vertical se designa como y , entonces y será la *coordenada de fila* del pixel (ver Figura 10).

Figura 10: El sistema de coordenadas de pantalla.



Note que en este sistema de coordenadas, el *número de filas crece hacia abajo en la pantalla* (y no hacia arriba, como quizás el estudiante esperaría dada la costumbre de los ejes cartesianos que normalmente se usan en matemáticas). Esto quiere decir que una fila que se encuentre más cerca del borde superior de la pantalla o ventana, tendrá un número *menor* que otra fila que se encuentre más abajo. El crecimiento en el número de columnas sigue la misma idea que los gráficos cartesianos normales (es decir, los números de columna crecen hacia la derecha en la pantalla).

El programa completo para generar la pirámide de la Figura 9 es el que sigue (y viene acompañando a esta Ficha en el proyecto [F14] *Piramide*):

```
from tkinter import *

def pyramid(canvas, x, y, r):
    if r > 0:
        # dibujar recursivamente el soporte inferior izquierdo...
        pyramid(canvas, x-r, y+r, r//2)

        # dibujar recursivamente el soporte inferior derecho...
        pyramid(canvas, x+r, y+r, r//2)

        # dibujar recursivamente el soporte superior izquierdo...
        pyramid(canvas, x-r, y-r, r//2)

        # dibujar recursivamente el soporte superior derecho...
        pyramid(canvas, x+r, y-r, r//2)

        # dibujar en (post-orden) la tapa o cima de la pirámide...
        square(canvas, x, y, r)

def square(canvas, x, y, r):
    left = x - r
    top = y - r
```



```
right = x + r
down = y + r

canvas.create_rectangle((left, top, right, down), outline='yellow', fill='blue')

def render():
    # configuracion inicial de la ventana principal...
    root = Tk()
    root.title("Piramide Fractal")

    # calculo de resolucioen en pixels de la pantalla...
    maxw = root.winfo_screenwidth()
    maxh = root.winfo_screenheight()

    # ajustar dimensiones y coordenadas de arranque de la ventana...
    root.geometry("%dx%d+%d+%d" % (maxw, maxh, 0, 0))

    # un lienzo de dibujo dentro de la ventana...
    canvas = Canvas(root, width=maxw, height=maxh)
    canvas.grid(column=0, row=0)

    # sea valor inicial de r igual a un duodécimo del ancho de la pantalla...
    r = maxw // 12

    # coordenadas del centro de la pantalla...
    cx = maxw // 2
    cy = maxh // 2

    # dibujar piramide centrada en (cx, cy) y el cuadrado mayor con lado = 2r.
    pyramid(canvas, cx, cy, r)

    # lanzar el ciclo principal de control de eventos de la ventana...
    root.mainloop()

if __name__ == '__main__':
    render()
```

El programa comienza con una instrucción *import* para dar acceso al contenido del módulo *tkinter*, que es el que básicamente contiene las declaraciones y funciones para manejar ventanas y gráficos:

```
from tkinter import *
```

La función *pyramid()* es la que realiza **el gráfico completo de la pirámide** usando recursión, pero veremos primero la forma de dibujar un cuadrado simple.

La función *square(canvas, x, y, r)* es la encargada simplemente de dibujar un cuadrado de bordes amarillos y pintado por dentro de color azul.

```
def square(canvas, x, y, r):
    left = x - r
    top = y - r

    right = x + r
    down = y + r

    canvas.create_rectangle((left, top, right, down), outline='yellow', fill='blue')
```

El parámetro *canvas* recibido por la función, es el lienzo o área de dibujo en donde debe graficarse el cuadrado (por el momento, no es necesario saber de dónde sale ese lienzo: simplemente, la función *square()* lo toma como parámetro y dibuja en él el cuadrado pedido).



Los parámetros x e y son las coordenadas de columna y fila (respectivamente) del pixel que queremos que sea el *centro* del cuadrado dentro del *canvas* donde será dibujado. Y el parámetro r es el valor de la mitad de la longitud del lado del cuadrado a dibujar (en pixels), por lo que la longitud total del lado será igual a $2*r$.

Las cuatro primeras líneas de la función *square()* calculan entonces las coordenadas de pixel de las esquinas superior izquierda e inferior derecha del cuadrado (asignando esos valores en las variables (*left, top*) y (*right, down*)) (asegúrese de entender la naturaleza de estos cuatro cálculos).

Finalmente, en la última línea de la función *square()*, se invoca a la función *canvas.create_rectangle()* para dibujar el cuadrado. Un elemento u objeto de tipo *Canvas* dispone de funciones (o métodos) para dibujar distintos tipos de figuras básicas o primitivas. En este caso, el método *create_rectangle(left, top, right, down, outline='yellow', fill='blue')* recibe como primer parámetro una tupla conteniendo los valores de las coordenadas de los pixels superior izquierdo e inferior derecho del rectángulo a dibujar. El parámetro *outline* (accedido por palabra clave) indica el color con el que será dibujado el borde o perímetro del rectángulo (aquí lo asignamos en amarillo ('yellow')). Y el parámetro *fill* (también accedido por palabra clave) indica el color con el que será pintado por dentro el rectángulo (y aquí lo asignamos en azul ('blue')). [2]

Note que esta función **no dibuja la pirámide**: sólo dibuja **un** cuadrado en las coordenadas centrales (x, y) que se le indiquen y con longitud de su lado igual a $2r$. Es la función *pyramid()* la que dibuja **toda la pirámide**, aplicando recursión e invocando tantas veces como sea necesario a la función *square()*.

La *definición recursiva* del concepto de "pirámide formada por cuadrados" puede enunciarse intuitivamente así:

"Una **pirámide formada por cuadrados de lado máximo $2r$** estará vacía si r es 0 (no se puede dibujar un lado de longitud 0), o bien contendrá un cuadrado de lado $2r$ en la cima que estará apoyado en cuatro soportes. **Pero estos cuatro soportes serán a su vez pirámides formadas por cuadrados cuyos lados serán de longitud máxima r .**"

Claramente esta definición es recursiva: el concepto de pirámide aparece en la propia definición. Pues bien: se trata de dibujar **una pirámide formada por cuatro pirámides menores (los soportes) y un cuadrado en la cima**. Y eso es exactamente lo que hace la función *pyramid(canvas, x, y, r)* (los parámetros tienen el mismo significado que en la función *square()*):

```
def pyramid(canvas, x, y, r):
    if r > 0:
        # dibujar recursivamente el soporte inferior izquierdo...
        pyramid(canvas, x-r, y+r, r//2)

        # dibujar recursivamente el soporte inferior derecho...
        pyramid(canvas, x+r, y+r, r//2)

        # dibujar recursivamente el soporte superior izquierdo...
        pyramid(canvas, x-r, y-r, r//2)

        # dibujar recursivamente el soporte superior derecho...
        pyramid(canvas, x+r, y-r, r//2)

        # dibujar en (post-orden) la tapa o cima de la pirámide...
        square(canvas, x, y, r)
```



La situación trivial se da cuando r es 0: en ese caso la función simplemente termina sin hacer nada. Pero si r es mayor a 0 (la longitud del lado del cuadrado es por lo menos $2*1 = 2$), entonces se procede al dibujo "desde abajo y hacia la cima", con cuatro invocaciones recursivas (una para cada soporte). Usamos recursión pues tenemos que dibujar *cuatro pirámides que sostengan a la pirámide mayor*...

Los valores x , y , r enviados como parámetro a cada instancia recursiva de la función `pyramid()` se calculan de forma que cada nueva pirámide se dibuje con diferentes coordenadas del centro y una longitud $2r$ para el lado del cuadrado en la cima progresivamente menor, dividiendo por 2 a r en cada invocación. Esta sucesión de divisiones en algún momento hará que se llegue a un valor de r que será igual a 0, y en ese momento se detendrá la recursión.

Observe un detalle: las cuatro invocaciones recursivas se hacen **antes** que la invocación a la función `square()`... lo cual tiene sentido, ¡ya que primero deben construirse las paredes antes de colocar el techo! Formalmente, como el proceso de dibujar el cuadrado se hace después que terminan las invocaciones recursivas, entonces tenemos un proceso de dibujo en *post-orden()* (u *orden posterior*). [5]

La última función que queda en el programa es `render()`, y es la encargada de preparar el contexto visual y gráfico:

```
def render():
    # configuracion inicial de la ventana principal...
    root = Tk()
    root.title("Piramide Fractal")

    # calculo de resolucion en pixels de la pantalla...
    maxw = root.winfo_screenwidth()
    maxh = root.winfo_screenheight()

    # ajustar dimensiones y coordenadas de arranque de la ventana...
    root.geometry("%dx%d+%d+%d" % (maxw, maxh, 0, 0))

    # un lienzo de dibujo dentro de la ventana...
    canvas = Canvas(root, width=maxw, height=maxh)
    canvas.grid(column=0, row=0)

    # sea valor inicial de r igual a un duodécimo del ancho de la pantalla...
    r = maxw // 12

    # coordenadas del centro de la pantalla...
    cx = maxw // 2
    cy = maxh // 2

    # dibujar piramide centrada en (cx, cy) y el cuadrado mayor con lado = 2r.
    pyramid(canvas, cx, cy, r)

    # lanzar el ciclo principal de control de eventos de la ventana...
    root.mainloop()
```

Prácticamente todas las novedades técnicas para la creación y control de la ventana principal y el `canvas` están en esta función [2]. La primera instrucción crea una variable que hemos llamado `root` con la función `Tk()`. Una variable de tipo `Tk` representa una ventana básica, con bordes definidos, barra de título y controles de minimización y cierre de la ventana. La variable `root` será entonces la que nos permitirá controlar lo que ocurre con la ventana donde se despliega el gráfico. Y la segunda instrucción simplemente asigna la cadena de caracteres que será visualizada en la barra de título de esa ventana:



```
# configuracion inicial de la ventana principal...
root = Tk()
root.title('Piramide Fractal')
```

Las siguientes dos líneas permiten básicamente averiguar la resolución (en pixels) de la pantalla en el momento de ejecutar el programa, para poder luego calcular en forma correcta las dimensiones de la ventana y, sobre todo, las coordenadas del pixel central:

```
# calculo de resolucion en pixels de la pantalla...
maxw = root.winfo_screenwidth()
maxh = root.winfo_screenheight()
```

La función `winfo_screenwidth()` retorna el ancho máximo en pixels admitido por la resolución actual de la pantalla, y la función `winfo_screehight()` hace lo mismo pero con la altura. Al momento de ejecutar este programa para preparar esta Ficha, la resolución era de 1366 pixels de ancho por 768 pixels de alto, y esos dos valores (o los que correspondan) se almacenan en las variables `maxw` y `maxh`.

La quinta línea del programa ajusta lo que se conoce como la *geometría de la ventana principal* (que como vimos, tenemos representada con la variable `root`):

```
# ajustar dimensiones y coordenadas de arranque de la ventana...
root.geometry('%dx%d+%d+%d' % (maxw, maxh, 0, 0))
```

La función `geometry()` permite ajustar el ancho y el alto de la ventana en cualquier momento, y también las coordenadas del pixel superior izquierdo desde donde debe mostrarse esa ventana. La función toma como parámetro una *cadena de caracteres* de la forma `'WxH+x+y'` en la que `W` es el ancho en pixels que queremos que tenga la ventana al mostrarse, `H` es la altura en pixels para la ventana, `x` es la coordenada de columna del pixel superior izquierdo donde queremos que aparezca, e `y` es la coordenada de fila de ese mismo pixel. A modo de ejemplo simple, si hubiésemos hecho algo como:

```
root.geometry('400x300+50+50')
```

entonces la ventana representada por la variable `root` tomaría un ancho de 400 pixels, una altura de 300 pixels, y sería visualizada con su esquina superior izquierda en el pixel de coordenadas (50, 50) de la pantalla.

El problema es que en muchos casos los valores que se quieren asignar están contenidos en variables y no vienen como constantes directas. En nuestro programa, por ejemplo, queremos que la ventana aparezca con un ancho tan grande como el que se tenga disponible en la resolución actual (y como vimos, tenemos ese valor almacenado en la variable `maxw`) y lo mismo vale para la altura (que tenemos en `maxh`). Si quisiéramos esos valores ancho y alto, y hacer aparecer la ventana desde el pixel (0, 0), entonces una invocación de la forma:

```
root.geometry('maxw x maxh + 0+0')
```

simplemente provocaría un error al ejecutarse: la cadena contiene letras donde Python esperaba encontrar dígitos...

La forma de tomar esos valores desde variables, consiste en usar *caracteres de reemplazo* (también llamados *caracteres de formato*) en la cadena. El par de caracteres `%d` se toma como si fuese un único carácter, y se interpreta como que en ese lugar debe ir el valor de



una variable de tipo entero (*int*) que se informa más adelante en la propia lista de parámetros de la función. En nuestro programa, la función se está invocando así:

```
root.geometry('%dx%d+%d+%d' % (maxw, maxh, 0, 0))
```

lo cual significa que el primer carácter **%d** (marcado en rojo en el ejemplo) será reemplazado por el valor de la variable **maxw** que es a su vez la primera en la tupla que aparece luego del signo % (de color negro). En forma similar, el segundo **%d** (azul) será reemplazado por el valor de la variable **maxh**, que es la segunda en la tupla, y así hasta el final.

Las dos instrucciones siguientes asignan una variable que hemos llamado *canvas* con un elemento u objeto de tipo *Canvas* para hacer allí los gráficos que necesitamos:

```
# un lienzo de dibujo dentro de la ventana...
canvas = Canvas(root, width=maxw, height=maxh)
canvas.grid(column=0, row=0)
```

La función *Canvas()* crea un objeto que representa un lienzo de dibujo. Esa función toma varios parámetros, y en este caso el primero es la variable *root* que representa nuestra ventana principal. Al enviar la ventana como parámetro, le estamos indicando al programa que el *Canvas* que estamos creando (variable *canvas*) estará contenido en esa ventana (variable *root*) y por eso todo lo que se dibuje en *canvas*, aparecerá en la ventana *root*.

Los otros dos parámetros que estamos enviando a la función *Canvas()* se están accediendo por palabra clave, e indican el ancho y el alto en pixels que debe tener el *canvas* dentro de la ventana. En este caso, simplemente hemos asumido que el *canvas* será el único elemento contenido en la ventana *root*, y le hemos fijado un ancho y un alto máximo (los valores de nuestras variables *maxw* y *maxh*). Por supuesto, esto puede ajustarse a voluntad del programador.

Cuando una ventana es creada, todos los elementos que se incluyan dentro de ella serán distribuidos en filas y columnas, como si el interior de la ventana fuese una tabla. La invocación a la función *grid()* que sigue a la creación del *canvas*, le dice al programa que el *canvas* debe ser ubicado dentro de la ventana en la "casilla" indicada por los parámetros *column* y *row* (en este caso, la casilla [0, 0]).

La instrucción que sigue sólo calcula el valor inicial para *r*, la mitad de la longitud del lado del cuadrado mayor que irá en la cima de la pirámide:

```
# sea un valor inicial de r igual a un duodécimo del ancho del area de dibujo...
r = maxw // 12
```

En este caso, se está tomando un valor igual a la duodécima parte del valor del ancho máximo para la ventana. El estudiante puede cambiar el 12 por otro valor y explorar los efectos que eso produce.

Las dos instrucciones que siguen simplemente calculan las coordenadas del centro de la pantalla:

```
# coordenadas del centro de la pantalla...
cx = maxw // 2
cy = maxh // 2
```

Estas coordenadas son importantes para nuestro programa, ya que le indican en qué posición del *canvas* de la ventana deberá estar centrada la vista de la pirámide, y a partir de esa posición la pirámide se irá construyendo hacia afuera.

La anteúltima instrucción invoca a la función *pyramid()* para dibujar la pirámide completa, cuyo funcionamiento recursivo ya hemos explicado más arriba:

```
# dibujar piramide centrada en (cx, cy) y el cuadrado mayor con lado = 2r.  
pyramid(canvas, cx, cy, r)
```

Sólo note que el parámetro *canvas* es el lienzo de dibujo que hemos creado dentro de la ventana, y que la función *pyramid()* lo recibe justamente para poder desarrollar en él nuestro gráfico.

Finalmente, la última instrucción es la que efectivamente pone todo a funcionar:

```
# lanzar el ciclo principal de control de eventos de la ventana...  
root.mainloop()
```

La ventana representada por la variable *root* se usa para invocar al método *mainloop()*, el cual hace que la ventana se muestre y quede a disposición del usuario. Toda acción que ese usuario aplique sobre la ventana (pasar el mouse sobre ella, minimizarla, presionar el botón de cierre, etc.) genera en el programa lo que se conoce como un *evento*, y el método *mainloop()* se encarga de tomar todos los eventos producidos por la ventana *root* y procesarlos adecuadamente. Por caso, es el método *mainloop()* el que cierra la ventana y da por terminado el programa cuando el usuario presione el botón de cierre ubicado arriba y a la derecha de la ventana.

9.] Pruebas y aplicaciones.

El programa que hemos mostrado en la sección anterior puede ser modificado de distintas maneras para producir diferentes (y muy interesantes) resultados gráficos. Nos permitimos mostrar más abajo algunas gráficas que hemos generado con algunas de esas variantes, y sugerimos al estudiante a que explore y aprenda en qué formas podría generar las mismas figuras efectuando distintos cambios en el código fuente original. Diviértase...

Figura 11: Gráficas producidas con modificaciones simples del programa [F14] Piramide [Parte 1]

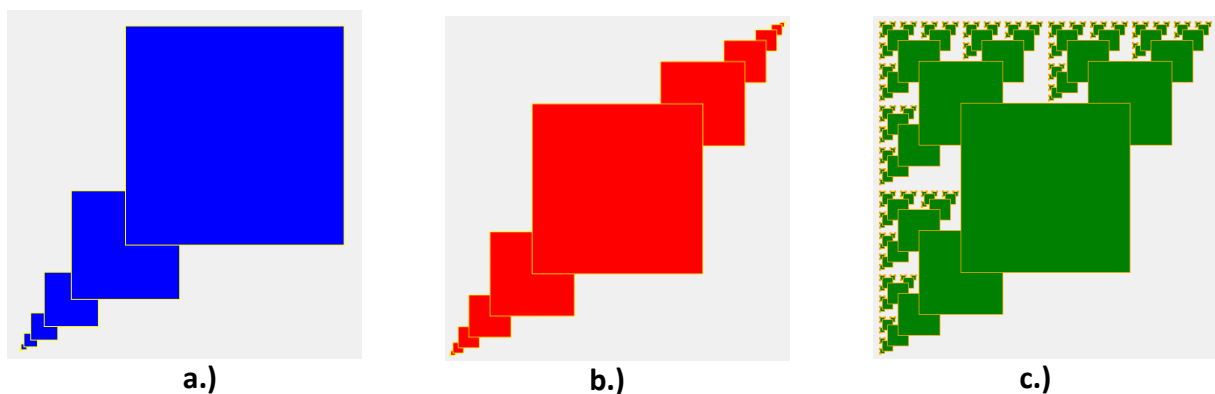


Figura 12: Gráficas producidas con modificaciones *no tan simples* del programa [F14] Piramide [Parte 2]

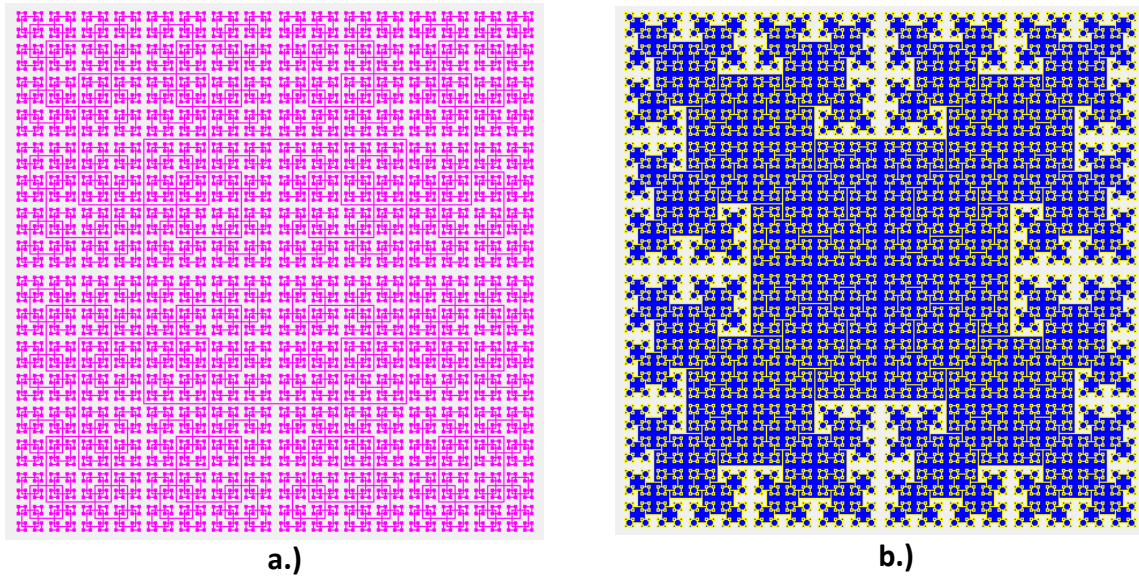
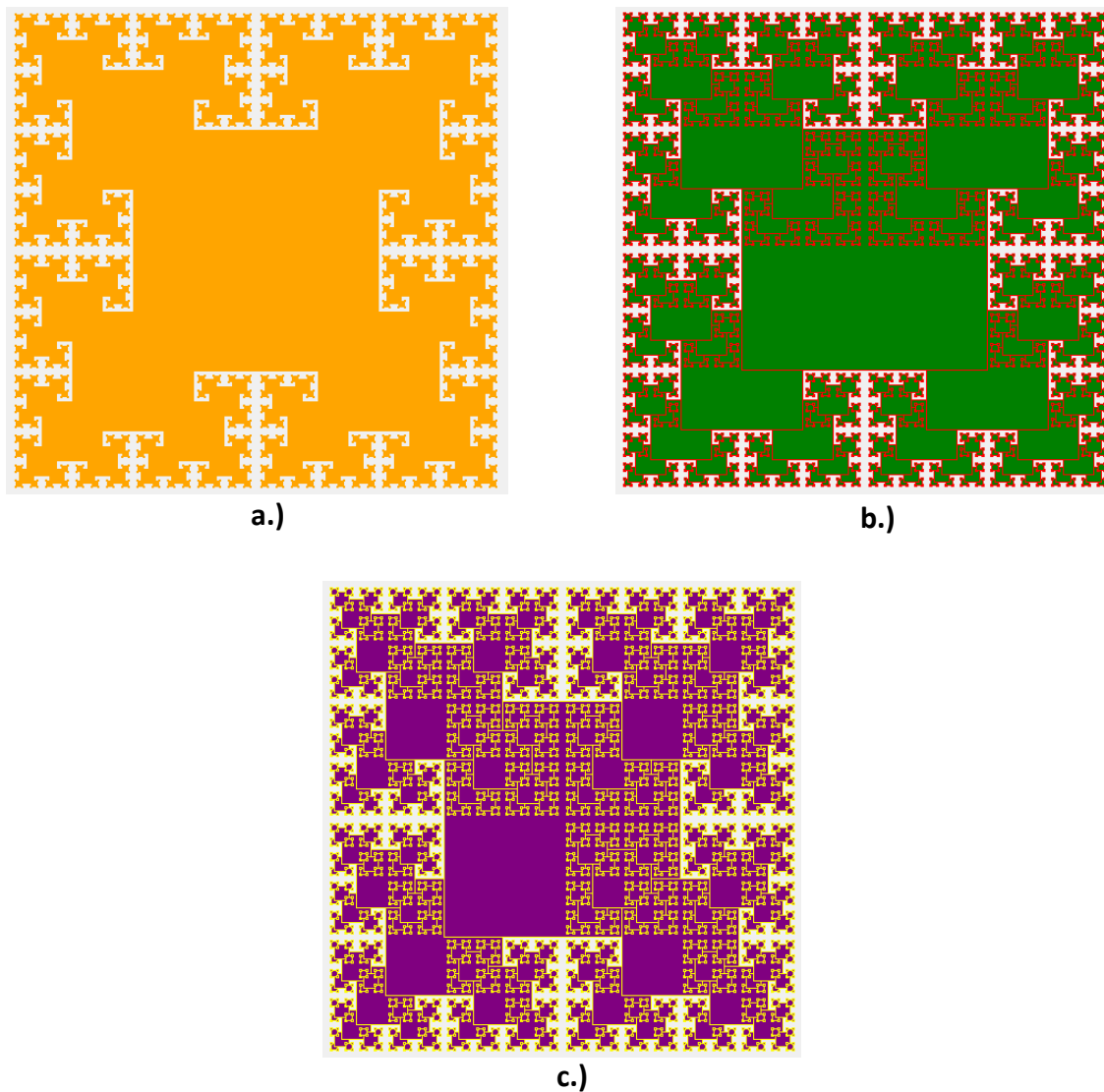


Figura 13: Gráficas producidas con modificaciones *no tan simples* del programa [F14] Piramide [Parte 3]





10.] La Función de Ackermann.

La *Función de Ackermann* es una función recursiva matemática que debe su nombre a quien la postuló en 1926: *Wilhelm Ackermann* [7]. La *Función de Ackermann* se basa en aplicar un operador simple (la suma) combinado con numerosas invocaciones recursivas, con la intención de lograr *un crecimiento muy rápido* en la magnitud de los valores calculados. Toda la explicación necesaria para comprender su uso y la forma de programarla, se expone en el problema que sigue:⁵

Problema 35.) *La Función de Ackermann originalmente planteada por Wilhelm Ackermann en 1926 ha sido ajustada a lo largo de los años hasta terminar siendo definida en la forma que se muestra aquí:*

Función de Ackermann

$$A(m, n) \begin{cases} = n + 1 & (\text{si } m = 0) \\ = A(m - 1, 1) & (\text{si } m > 0 \text{ y } n == 0) \\ = A(m - 1, A(m, n - 1)) & (\text{si } m > 0 \text{ y } n > 0) \end{cases}$$

(m, n enteros
m >= 0, n >= 0)

Se pide desarrollar un programa que permita calcular el valor de la Función de Ackermann para distintos valores de m y n.

Discusión y solución: La función originalmente planteada por *Ackermann* tenía tres parámetros en lugar de dos. Como dijimos, a lo largo de los años se han ido sugiriendo variantes y modificaciones que terminaron conformando una *familia* o *serie de funciones de Ackermann*, basadas en estructuras similares. La definición que mostramos en el enunciado es una variante que fue propuesta por *Rózsa Peter* y *Raphael Robinson*, aunque es común que se la cite como si fuese la original de *Ackermann* [8]. En algunos contextos, la variante de *Peter* y *Robinson* se conoce también como la *Función de Ackermann-Peter*. La función está definida directamente en forma recursiva, por lo que su programación también es directa y no ofrece mayores problemas (ver el proyecto *F[14] Ackermann* que acompaña a esta Ficha):

```
def ackermann(m, n):  
    if m == 0:  
        return n + 1  
    elif n == 0:  
        return ackermann(m - 1, 1)  
    else:  
        return ackermann(m - 1, ackermann(m, n - 1))
```

Técnicamente hablando, el problema está resuelto. Sin embargo, conviene tomarse un tiempo para estudiar esta función y sus propiedades. Lo primero que se observa es que sólo la primera parte de la definición es *no recursiva*: si $m = 0$ entonces $A(0, n) = n + 1$. Esto significa que lo que sea que termine calculando, será el resultado de aplicar ese único operador directo (la suma), para añadir 1 al valor de n . El siguiente programa (incluido en el mismo proyecto *F[14] Ackermann*) calcula diversos valores para la función haciendo que m tome todo valor posible en el intervalo $[0, 3]$ y n tome valores del intervalo $[0, 6]$:

⁵ La fuente esencial de la explicación que sigue sobre la *Función de Ackermann*, es la *Wikipedia* (especialmente la versión en inglés): https://en.wikipedia.org/wiki/Ackermann_function. También hemos usado una referencia básica contenida en el libro "*Estructuras de Datos con C y C++*" de *Yedidyah Langsam* (y otros), así como alguna cita del libro "*Estructuras de Datos en Java*" del ya citado *Mark Allen Weiss*.



```
def test():
    for m in range(4):
        for n in range(7):
            ack1 = ackermann(m, n)
            print('Ackermann(', m, ', ', n, '): ', ack1, '\t-\t', sep='', end='')
            print()
```

```
test()
```

Podría parecer entonces que la función entregará resultados que no serán de valor muy alto, ya que todos los cálculos se hacen en base a sumar 1 al valor actual de n en cada instancia recursiva. Pero la realidad es que para valores combinados de m y n relativamente bajos, la función llega a resultados asombrosamente elevados. Mientras el valor de m se mantenga menor a 4 y el valor de n se mantenga menor a 7, la función devuelve valores de magnitud aceptable, como se puede ver en la siguiente salida generada por el programa anterior:

A(0,0): 1	A(0,1): 2	A(0,2): 3	A(0,3): 4	A(0,4): 5	A(0,5): 6	A(0,6): 7
A(1,0): 2	A(1,1): 3	A(1,2): 4	A(1,3): 5	A(1,4): 6	A(1,5): 7	A(1,6): 8
A(2,0): 3	A(2,1): 5	A(2,2): 7	A(2,3): 9	A(2,4): 11	A(2,5): 13	A(2,6): 15
A(3,0): 5	A(3,1): 13	A(3,2): 29	A(3,3): 61	A(3,4): 125	A(3,5): 253	A(3,6): 509

Si el valor inicial de m es 0, la función directamente retorna $n + 1$, y eso produce que la primera fila de la tabla anterior simplemente contenga la progresión de los números naturales para los resultados. Las dos filas que siguen se mantienen con valores de salida pequeños. Pero la cuarta fila ya comienza a producir resultados bastante mayores. Para $A(3,5)$ el resultado ya es 253 y para $A(3,6)$ se obtiene un 509. Evidentemente, cuando los valores combinados de m y n comienzan a crecer, la función realiza una cantidad muy alta de invocaciones recursivas, y eso hace que el valor de salida sea muy alto también (aunque se obtenga sólo con la acción de sumar 1...). Si intentamos calcular $A(3,7)$ el programa se interrumpe por *overflow de stack* sin llegar a mostrarnos el resultado: la cantidad de instancias recursivas es tan grande que no puede ser soportada por el *Stack Segment*.

Lo anterior lleva a que nos preguntemos si la función está efectivamente bien planteada, o si por el contrario, pudiera estar entrando en una situación de recursión infinita que sea la responsable del *overflow de stack*. Al fin y al cabo, no parece evidente que $A(m,n)$ llegue a terminar de calcularse alguna vez para valores de m y n bastante mayores que 0, ya que la segunda parte de la definición de la función hace una llamada recursiva, pero la tercera hace algo más complejo aún: *una invocación recursiva compuesta con otra* (la primera toma como parámetro al resultado de la segunda, haciendo una *composición de funciones*).

Sin embargo, un análisis detallado del comportamiento de la función nos lleva a concluir que la función está bien planteada y no produce una regresión infinita: si m y n son diferentes de cero, ambos son restados en 1 en algún momento durante la cascada recursiva, pero se comienza restando 1 a n . Cuando el valor de n llega a ser 0, comienza desde allí a restarse 1 a m , lo que eventualmente llevará a $m = 0$ y a la detención de la cascada recursiva. El tema es que hasta que eso ocurra, se creará una inmensa cantidad de instancias recursivas que hará que los valores retornados sean cada vez mayores, y provocando también el rebalsamiento de *stack*.

Para terminar de comprender la magnitud de los valores que llega a calcular la función, note que $A(4,0)$ vale 13. El valor $A(4,1)$ ya pasa a ser 65533 (aunque nuestra función posiblemente ya no pueda calcularlo por producirse un *overflow de stack*). Y el valor $A(4,2)$ es tan inmenso que no puede escribirse sino en forma exponencial: ese valor es igual a $2^{65536} - 3$ (de hecho, este número resulta ser mayor que u^{200} donde u es la cantidad total de partículas contenidas



en el universo). Y se pone mucho peor: los valores de la función para $m > 4$ y $n > 1$ son tan extravagantemente enormes que no podrían escribirse nunca en forma normal, ya que la secuencia de dígitos que conforma a cada uno de esos números *no cabe en el universo conocido* (y por supuesto, ni nuestro programa ni ningún otro podría siquiera comenzar a calcular esos valores, ya que mucho antes de llegar al final se produciría un overflow de stack).

Para finalizar, digamos que la *Función de Ackermann* tiene ciertos usos prácticos: por lo pronto, debido a su naturaleza profundamente recursiva, puede utilizarse para testear la habilidad de un compilador en optimizar un proceso recursivo. En general, cuando se compila un programa recursivo, el compilador intenta eliminar la recursión y producir una versión compilada iterativa, que será más eficiente en cuanto a uso de memoria y posiblemente en velocidad de ejecución. La *Función de Ackermann* puede ser entonces uno de los desafíos que se le imponen a un compilador cuando esté en etapa de prueba (o *benchmarking*). Además, dado que la *Función de Ackermann tomada para $m = n$* (o sea: $A(n, n)$, que a su vez puede simplificarse como $A(n)$) crece de forma tan violenta, entonces su inversa (es decir, la función que a cada número y calculado por $A(n)$ le asigna el mismo valor n de partida) normalmente designada con la *letra griega alfa*: $\alpha(n) = A^{-1}(n)$ aumenta de manera sorprendentemente lenta. De hecho, la función $\alpha(n)$ retorna valores menores que 5 para *prácticamente cualquier valor de n* que se tome como parámetro. Esta propiedad de la *inversa de Ackermann* de crecer tan lentamente, suele usarse en el contexto del *análisis de algunos algoritmos* que se sabe que tienen un tiempo de ejecución que crece de forma muy lenta a medida que aumenta el número de datos.

Bibliografía

- [1] V. Frittelli, Algoritmos y Estructuras de Datos, Córdoba: Universitas, 2001.
- [2] Python Software Foundation, "Python Documentation," 2015. [Online]. Available: <https://docs.python.org/3/>. [Accessed 24 February 2015].
- [3] R. Sedgewick, Algoritmos en C++, Reading: Addison Wesley - Díaz de Santos, 1995.
- [4] M. A. Weiss, Estructuras de Datos en Java - Compatible con Java 2, Madrid: Addison Wesley, 2000.
- [5] M. Pilgrim, "Dive Into Python - Python from novice to pro," 2004. [Online]. Available: <http://www.diveintopython.net/toc/index.html>. [Accessed 6 March 2015].
- [6] V. Frittelli, R. Teicher, M. Tartabini, J. Fernández and G. F. Bett, "Gestión de Gráficos (en Java) para Asignaturas de Programación Introductiva," in Libro de Artículos Presentados en I Jornada de Enseñanza de la Ingeniería - JEIN 2011, Buenos Aires, 2011.
- [7] Wikipedia, "Ackermann function," 2015. [Online]. Available: https://en.wikipedia.org/wiki/Ackermann_function.
- [8] Y. Langsam, M. Augenstein and A. Tenenbaum, Estructura de Datos con C y C++, México: Prentice Hall, 1997.